Three-person demo script

Multisensor Engine Diagnostics | 10 minutes | Replace Presenter 1/2/3 with your names before rehearsing.

One-line pitch: We built a system that listens to an engine the way a mechanic does, with a microphone, a vibration sensor and the engine computer's data, and tells you which fault is developing, in which cylinder, and why. It does this seconds before the car's own warning system.

Time	Speaker	Screen action
0:00-3:00	Presenter 1	Problem, scope, dashboard tour, healthy engine (preset 1)
3:00-7:00	Presenter 2	Play preset 13; explain the signal-processing and ML pipeline
7:00-10:00	Presenter 3	Evidence, cylinder, ECU comparison, Diagnose and Report, results and close

Presenter 1 - Why we built it

0:00-3:00 | Stand at the dashboard home screen, red banner visible

"A check-engine light tells you something is wrong after the damage has started. A misfire can wreck a catalytic converter. Knock, the engine pinging under load, can crack a piston. Our question is simple: can we hear these faults early, tell which one it is, and show the evidence?"

"The hard part is that an engine is never quiet and never constant. Change the speed or the load and the sound changes completely. So a louder signal does not mean a fault. Our system has to know what normal looks like at this exact speed and load, and only then call something abnormal."

"We covered three modeled faults: **misfire**, where a cylinder fails to fire; **knock**, uncontrolled combustion that makes sharp bursts; and **valve-clearance wear**, a ticking from the valve train. And we handle them **at the same time**, because real engines do not break one thing at once."

Screen cue: point at the red "SYNTHETIC DATA - RECORDED REPLAY" banner.

"Everything you will see today is synthetic. We wrote a MATLAB engine simulator that produces microphone, vibration and engine-computer data for a four-cylinder engine, and the dashboard replays recorded runs. We say that up front, and the banner never leaves the screen."

Screen cue: select preset 1, "Normal operation with a load step", press Play.

"This is a healthy engine the model has never seen. At about nine seconds the load steps up. The sound changes, but the decisions stay normal and no alert fires. That is the first test: do not cry wolf."

Handoff: "Presenter 2 will play a run with three faults and show how raw sound becomes a diagnosis."

Presenter 2 - How the analysis works

3:00-7:00 | Select preset 13 and press Play. If the dashboard stalls, switch to preset 11 or the backup recording.

Screen cue: press Play on "13. Three faults, three independent views". Let it run under the next paragraph.

"We built this in three layers. First, a **MATLAB crank-angle event simulator**. It models every combustion event in a four-stroke engine and renders a **24 kHz microphone signal**, a **25.6 kHz accelerometer signal** and six ECU telemetry channels, plus a **60-2 crank trigger wheel**, a **cam-position pulse**, and **exhaust and cylinder-head temperature per cylinder**. That is real automotive instrumentation, not a toy."

"Second, a **Python pipeline**. Every run passes a written **data contract** and gets a **SHA-256 hash**, so results are reproducible bit for bit. Then the replay engine streams the recording in **40 millisecond packets**, like live sensor data, and we tested that no decision ever uses a sample from the future."

"Every half-second we take the last **two-second trailing window** and compute **31 interpretable features**: **Welch power spectral density**, which shows how much energy sits at each frequency; **kurtosis** and **crest factor**, which measure how spiky a signal is; and a **Hilbert-transform envelope** of the 5 to 10 kilohertz band, which is where knock bursts live. The MATLAB and Python versions of these features agree to nine decimal places."

"The model has two parts. A **normal baseline**: for each speed-and-load bin we store what healthy looks like using **robust median and MAD statistics**, then score each window by its **robust z-score**, meaning how many normal-sized deviations it is away. Then a **calibrated RBF support vector machine**, one detector per fault, so two or three faults can be named at once. **Sigmoid calibration** on separate engines means a 90 percent confidence behaves like 90 percent."

Screen cue: point to the alert timeline and the evidence table.

"Alerts need **two consecutive confident decisions** and clear only after three normal ones, so the display does not flicker. Bad data, like an audio dropout, produces a quality state and no guess."

"Third, the part we are proudest of. A **stroke-synchronous feature set** uses the cam pulse to place every combustion in **crank angle**, so the system can say **which cylinder**. A **rotation-equivariant conditional-logit localizer** picks it out. Then we cross-check with **three independent paths that share no learned weights**: **crank-speed analysis** from the trigger wheel, a simulated **OBD-II misfire monitor** as the industry benchmark, and an **EGT/CHT thermal path**: a cold cylinder means misfire, a hot head means knock."

Flow to say aloud: MATLAB simulator → 24 kHz audio, 25.6 kHz vibration, telemetry, crank wheel, temperatures → data contract + SHA-256 → 40 ms packet replay → 2 s windows, 31 features → normal baseline + calibrated SVM per fault → persistent alerts → cylinder localizer → crank, ECU and thermal cross-checks → dashboard.

Our main technology choices

Technology	Purpose in this project
MATLAB + Signal Processing Toolbox	Crank-angle engine simulator and sensor rendering
Python, SciPy, scikit-learn	DSP, features, calibrated RBF SVM, replay engine
Streamlit + Plotly	Dashboard: spectrogram, timeline, evidence, 3D engine
Google Gemini (optional, free tier)	Ranks possible causes in Diagnose; rule-based fallback if offline

Handoff: "Presenter 3 will show what an engineer sees when an alert fires, and how we did against the car's own monitor."

Presenter 3 - Evidence, results and close

7:00-10:00 | Preset 13 is at the end of its run. Open the evidence and the engine profile.

"Here is what matters to the person fixing the engine. Every alert comes with evidence: the three biggest deviations from this engine's own normal at this speed and load. It is evidence to inspect, not a verdict. We never claim root cause."

Screen cue: with truth overlay off, point to the three alerts, then turn the overlay on.

"Only now do we reveal the simulator's ground truth. Misfire on cylinder 1 at 8.9 seconds, valve fault on cylinder 2 at 17.5, knock on cylinder 2 at 21.5. Our alerts landed at 10.0, 21.0 and 22.5 seconds, and the model never saw that truth. Across all held-out test runs, the localizer named the right cylinder in 71 of 71 alert episodes."

"Now the comparison that counts. For misfire, we benchmarked against a simulated version of the ECU's own monitor. On held-out engines, our acoustic alert came a median **4.5 seconds earlier** than the ECU's code, and in 8 more misfire intervals the ECU never set a code before the run ended. The ECU cannot see knock or valve faults at all. We caught 20 of 20 knock and 15 of 16 valve intervals."

Screen cue: open the Diagnose and Report tabs on profile DEMO-057 ("Demo engine (three faults)").

"We also built an engine profile. Each replay and each technician note is saved to that engine's history, so Diagnose can rank possible causes and next steps from the history, and Report exports a PDF. Gemini only helps write the cause ranking. The detection is entirely our own models."

"We report what worked and what did not. On held-out virtual engines, the multi-fault model scores a window macro-F1 of **0.884**, against 0.621 for a single-label model on the same windows. It alerted every sustained fault in **4 of 4 three-fault runs** and **10 of 12 two-fault runs**. Processing takes 39 milliseconds per decision at the 95th percentile."

"The weak spots: valve faults, with 0.62 window recall; mild faults; and high noise at extreme RPM, where we mostly fail. Requiring the head-temperature confirmation cut knock false alerts from 0.30 to zero per minute, at the cost of recall and latency. All of this is on four test engines and synthetic data, so it does not prove real-engine accuracy. Real recordings are our next step."

> **Closing line:** "Our goal is to catch engine faults earlier than the warning light, name the cylinder, and keep the evidence and the uncertainty visible."

Live demo safety net

Close MATLAB; the demo runs from exported files only. Start the dashboard with *uv run --all-extras streamlit run dashboard/app.py*, open localhost:8501, speed 1x, truth overlay off. Keep the backup recording ready. If the dashboard stalls, say: "The replay is running locally; while it loads, here is a run from our recorded set." Do not call the data live or real.

Judge questions - quick answers

Answer directly, then show the relevant screen or evidence if asked.

Question	Short answer
Is this real engine data?	No. It is synthetic data from our MATLAB simulator, labeled on screen. Real-engine accuracy is not established.
Why not just a louder-than-normal threshold?	Sound changes with speed and load. We compare each window to normal for that exact speed and load bin.
Why an SVM and not deep learning?	It runs on a CPU in milliseconds and every feature is interpretable. A CNN comparison is future work.
Are you just reading RPM and load?	No. A model given only RPM, load and throttle scored 0.199 macro-F1, which is chance.
How do you know which cylinder?	A cam pulse places each combustion in crank angle. The localizer was right in 71 of 71 alert episodes.
Does it beat the ECU?	Against our simulated OBD-II monitor, acoustic misfire alerts came 4.5 s earlier at the median. The ECU also has no knock or valve capability.
Is the wear indicator a remaining-life prediction?	No. It tracks an accelerated simulated valve fault, not real wear.
What does the AI in Diagnose do?	Optional Gemini ranks possible causes from history. Without a key, a rule-based version runs. Detection never uses it.
What fails?	Valve faults (0.62 window recall), mild faults, and high-noise, high-RPM conditions.
What would you do next?	Real recordings, stronger valve signatures using the crank wheel, and training on wider noise and RPM ranges.

If a run misses an obvious fault: "That is a real limitation, and we report it. We would label this run, measure the miss, and use it in the next improvement cycle."

Three points everyone should remember

1. Say **synthetic data on held-out virtual engines**, not "real engines".
2. Say **evidence for an engineer to inspect**, not "we diagnosed the root cause".
3. Say **our own models detect faults; Gemini only helps rank causes**, not "AI listens to the engine".